openAG Standard (Normative)

Status: Normative draft — design-only. openAG has zero code and zero standard artifacts yet; AMS is frozen until openAG reaches parity. This document is the data-model contract: the entity model, the promise algebra, the axis + valence system, the gap taxonomy, the oracle set, and the NN-compilable graph-export contract. It is the normative companion to:

- `docs/openAG/engine-architecture.md` — the engine (operators, seam, components).
- `docs/openAG/tooling.md` — the CLI + MCP surface.
- `docs/openAG/cyber-physical-extensions.md` — additive domain profiles, the A8 Spatial axis, the twin promise vocabulary, and the 5th oracle; the normative companion for multi-disciplinary / digital-twin graphs.
- `docs/plans/2026-10-04-openAG-graph-native-engine-design.md` — formal core.
- `docs/plans/2026-10-04-openAG-entity-model.md` — gen-0 entity ledger.
- `docs/plans/2026-10-04-openAG-attribute-corpus.md` — provenance ledger.
- `docs/plans/2026-10-04-openAG-pipelines.md` — B-ROUND + B0–B7 pipelines.

Date: 2026-10-12. Conformance language: MUST / SHOULD / MAY per RFC 2119.

> Revision 2026-10-12 (cyber-physical extensions). This revision adds the hooks required to capture multi-disciplinary cyber-physical systems and serve as a digital-twin backbone: an opt-in domain-profile mechanism and `Component.subkind` (§2), an eighth (A8 Spatial) axis (§3), a twin promise vocabulary (§4.7), a 5th (metrology/CAD) oracle (§6), and conformance items 9–12 (§10). All additions are additive and opt-in — a software-only graph that uses none of them is unchanged. The normative detail lives in `docs/openAG/cyber-physical-extensions.md`; this document carries only the anchors.

---

1. Scope and purpose

openAG is a graph-native architecture engine whose single mission (F0) is:

> Sustain a verified architecture graph of a target system by promise-discharge.

The graph is the single source of truth. Every architectural claim is a promise attached to a node; a claim is true only when the promise is grounded in typed evidence; everything not grounded is an explicit typed gap. There are no silent omissions (gap-honesty, §7).

This standard governs the shape of the graph and the meaning of its promises. The mechanics of growing the graph (operators, scheduling, the neurosymbolic seam) are normative in `engine-architecture.md`; this document defines what a conformant graph and promise MUST look like.

---

2. Entity model

A conformant openAG graph is a typed, directed multigraph. Node kinds:

kind	role
System	the L0 root; owns the mission promise (F0).
Capability	a functional block the system provides (F-block).
Behavior	a use case / workflow / operational sequence.
Interface	an API / protocol / data exchange (carries two promises, §5.3).
Component	a deployable/allocatable unit (module, package, service).

kind	role
Actor	an external agent or fact-source oracle (§6).
Constraint	a non-functional rule or budget.
Layer	an architectural tier with an allowed dependency direction.
Promise	first-class node — a single `(owner, attribute)` claim.
Evidence	first-class node — a typed grounding for a promise.
I/O entity	owned outputs: Artifact, Journal, WorkOrder, Job, Proposal, Note, Lesson, Generation, Decision.

Promotion rule (normative). Promises, Evidence, and every I/O entity MUST appear as first-class nodes in the graph (not inline attributes), so that ownership, provenance, and grounding are themselves queryable and NN-compilable (§9).

Conservation laws (normative).

- A1 **`contains`** is a strict partition. Every Component belongs to exactly one parent along the A1 abstraction axis. A module is the partition atom. A cross-cutting concern MUST escape the partition via a dedicated shared Component or the `LAY-CrossCutting` layer — it MUST NOT be duplicated across partitions.
- A2 **`realizes`** is many-to-many free. A Component MAY realize many Capabilities and a Capability MAY be realized by many Components.

Domain profiles (opt-in). The node vocabulary above is software-flavored. A graph MAY activate one or more domain profiles — `controls | mechanical | electrical | software` — declared as a list in `meta.domain_profiles`. Each profile extends the vocabulary with discipline- specific subkinds, attributes, and validation rules. Every Component MAY then carry a `subkind` drawn from the union of active profiles' subkind vocabularies (and an optional `discipline` tag). Profiles are fully specified in `cyber-physical-extensions.md` §2; a graph that declares no profiles behaves exactly as this section defines.

---

3. The eight decomposition axes

Every relationship and every promise points along exactly one axis. The 23 AMS `RelationType` values map onto A1–A7 with zero loss (crosswalk in the entity-model appendix §11); A8 is net-new with the cyber-physical extensions.

axis	name	governs
A1	Abstraction	`contains` hierarchy; roll-up (↑) and flow-down (↓).
A2	Realization	`realizes` / `exposes` / `consumes` allocation.
A3	Boundary	`depends-on`, interface exposure/consumption.
A4	Governance	`constrained-by`, `allocated-to`, policy.
A5	Flow	`produces` / `subscribes-to` / `transforms` data & events.
A6	Lifecycle	`supersedes` / `migrates-to`; diachronic Generation lineage.

axis	name	governs
A7	Evidence	`has-promise` / `verified-by`; grounding.
A8	Spatial (opt-in)	physical placement & connection: `mounted-on` / `adjacent-to` / `mates-with` / `connected-at` / `routed-through`; coordinate frames. See `cyber-physical-extensions.md` §3.

A8 is orthogonal to A1: A1 `contains` is logical/abstraction containment (a strict partition); A8 is physical placement. Keeping them distinct is a core design assertion of the cyber-physical extensions. A1–A7 are mandatory; A8 is active only when a graph needs geometry/placement.

---

4. The promise algebra

4.1 PromiseSpec

A Promise node extends the AMS v3 `PromiseSpec` (`promise-graph.schema.json`) with two net-new openAG fields:

```
Promise {
  attribute        # the claimed attribute name (open vocabulary)
  content          # the claim text / value
  owner_entity     # the node this promise is attached to
  delivered_from   # provenance of the content
  verified_by      # evidence ref(s) that ground it
  status           # open | delivered | verified | typed-gap
  gap_type         # non-null IFF status == typed-gap
  provenance       # typed provenance tag(s)
  axis             # NET-NEW: A1..A8 — which axis the promise points along
  valence          # NET-NEW: the per-kind rule that DEMANDED the promise
}
```

- **`axis`** anchors the promise to one of the eight axes (§3).
- **`valence`** records why the promise exists — the kind-rule that demanded it (e.g. "every Capability MUST carry ≥1 `realizes` or `gap:unrealized`"). Valence is the declarative type-signature floor (§4.3).

4.2 Promise status

`PromiseStatus ∈ {open, delivered, verified, typed-gap}`:

- open — demanded by valence, not yet filled.
- delivered — content exists (design/discussion/reservoir) but not yet grounded in A7 runtime/observational evidence.
- verified — grounded in typed evidence (§4.4).
- typed-gap — nothing anywhere; emitted as `gap:<class>` (§7).

4.3 The four attribute end-states (normative)

Orthogonal to status (the sourcing axis) is valence (the verifiability axis). Crossing them, every attribute MUST resolve to exactly one of four terminal end-states:

end-state	verifiable?	sourced?	gap?	meaning
verified promise	yes	grounded	no	discharged against A7 evidence.
reservoir (open)	yes	content, no owner	no (yet)	promised, awaiting a lower-level owner (`axis: A1↓`).
plain attribute	no	yes	no	non-verifiable prose; retained-but-untracked; a legitimate terminal non-gap.
typed-gap	yes	nothing	yes	the only gap.

Rules.

- The valence floor is a declarative type-signature, not a schedule.
- `grow` MAY propose content above the floor into the reservoir.
- `ground` MUST promote only what verifies to verified.
- Inherently non-verifiable content MUST settle as a plain attribute — kept as prose, never tracked as an open promise, never counted as a gap.
- A reservoir item crosses to an owner via θ-crosswalk by content σ (`reconcile` + similarity; §4.5).
- Only **`typed-gap`** is a gap. Conformant tooling MUST NOT report plain attributes or reservoir items as gaps.

4.4 Evidence and grounding

An Evidence node is typed:

evidence type	grounds	produced by oracle
static	contracts, type signatures, schema (left-of-Vee).	code-scan, model-self
observational	journaled runtime facts, SI&L, benchmarks.	runtime, human-dialogue
executable	oracle runs, parity checks (right-of-Vee).	runtime

A promise is grounded iff ≥1 Evidence node is linked via `verified-by` (A7) and the grounding passes the axis-appropriate check (structural → symbolic, semantic → neural; §6).

4.5 Identity binding (θ-crosswalk)

Re-binding is fluid until grounded, then sticky. `reconcile` + a similarity score σ is the single binding operator for module identity, attribute-to-owner crosswalk, and cross-generation identity:

σ band	action
σ ≥ 0.85	bind — same identity.
0.60 ≤ σ < 0.85	ambiguous — emit `gap:ambiguous-identity`.
σ < 0.60	new — distinct identity.

4.6 Risk is intrinsic (not an attribute)

Every promise carries a latent risk = P(won't ground) × consequence. Risk MUST NOT be stored as a separate attribute or entity: it collapses into the VoI machinery (`engine-architecture.md` §VoI) where `consequence` is the value term and `P(won't ground)` is the confidence term. The risk register is a VIEW over the gap register sorted by consequence, surfaced at gate L2.

4.7 Twin promise vocabulary (opt-in, **`axis: A8`** / **`A7`**)

When a graph serves as a digital-twin backbone, components carry three families of promises whose `content` follows a typed shape. These are ordinary promises (same status / valence / grounding rules) — only their `content` is structured:

family	axis	attributes	grounds
geometry	A8	`geometry_ref` (URI), `pose`, `bounding_box`, `coordinate_frame`.	shape & placement.
kinematic	A8 + A5	`kinematic_chain`, `motion_profile`, `reach_envelope`.	articulated motion (carried by Behavior nodes).
live-binding	A7	`live_state` — a dynamic promise (§5.2) carrying a protocol/tag binding descriptor.	runtime state, grounded only by the runtime oracle.

Heavy payloads (meshes, point clouds) MUST NOT be inlined — they are referenced by URI only, preserving the `CON-N2` lossless export (§9). Full shapes are normative in `cyber-physical-extensions.md` §4. The model merely captures twin information; rendering, simulation, and live wiring are downstream and out of scope.

---

5. Interfaces and integration

5.1 Interface subkinds

`Interface.subkind ∈ {http, cli, message, rpc, library}` (software profile; other domain profiles leave it unset). Endpoint promotion generates deterministic Interface ids.

5.2 Static + dynamic promises (normative)

Every Interface (and, generalizing, every Behavior) MUST carry two grounding promises:

- a static promise — the contract / type signature — left-of-Vee, grounded by `reconcile` + schema/model-check against exposer and consumer;
- a dynamic promise — observed runtime behavior — right-of-Vee, grounded by the runtime oracle (§6).

5.3 Integration as promise-discharge

Integration is grounding the dynamic promise — there is no separate integration mechanism or deferred right-of-Vee phase. Consequences:

- An ungrounded dynamic promise is **`gap:integration-mismatch`**.
- Failure localizes to the interface's own promise; connected components keep their groundings (non-monotonicity dissolved).

- “Component A was wrong” is an under-specified promise superseded → a diachronic **`Generation`** change (A6), not a synchronic retraction.

---

6. The fact-source oracles

Grounding is mixed per-axis and arbitrates the oracles below, all VoI- arbitrated (no privileged axis). The base set is four; the cyber-physical extensions add a candidate 5th (metrology/CAD) oracle for A8 geometry:

oracle	actor	grounds	mode
code-scan	Target System	structural facts (modules, imports, endpoints).	symbolic
human-dialogue	Modeler / Auditor	intent, goals, acceptance.	neural / human
model-self	the graph reading itself	derived/rolled-up facts.	symbolic
runtime / composition-execution	Runtime	dynamic (right-of-Vee) interface + behavior promises.	executable
metrology / CAD (candidate, opt-in)	CAD system / laser scanner / survey	A8 geometry, pose, dimensions.	static-evidence

`ground` MUST route structural axes → symbolic checks and semantic axes → neural checks. Whether the 5th oracle is genuinely distinct or is the code-scan oracle generalized to “artifact-scan” is an open question (`cyber-physical-extensions.md` §5); until resolved, geometry promises ground via static evidence tagged `provenance: cad` / `provenance: metrology`.

---

7. Gap taxonomy (normative)

Gaps are sourced from the three verification classes plus the VoI economic model. A conformant engine MUST emit one of these — never a silent omission — whenever a demanded promise cannot be grounded within budget.

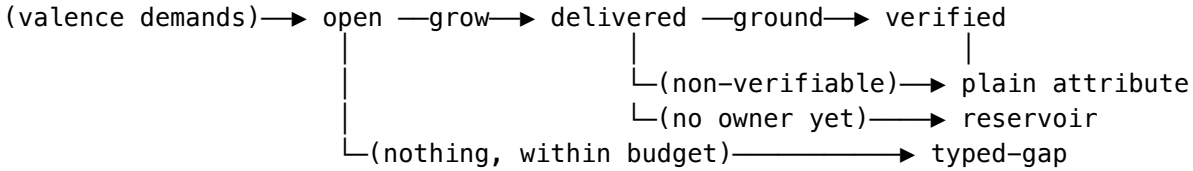
gap	source class	meaning
`gap:ungrounded`	existence	node has no A7 evidence.
`gap:missing`	existence	required promise absent.
`gap:unrealized`	existence	capability with no `realizes`.
`gap:orphan`	existence	component with no `realizes`-back.
`gap:dangling`	existence	interface missing exposer/consumer.
`gap:disconnected`	existence	actor with no consumes/triggers.
`gap:integration-mismatch`	existence (runtime)	dynamic (right-of-Vee) promise ungrounded.
`gap:violation`	prohibition	forbidden edge/state present.
`gap:unevaluable`	prohibition	constraint lacks a machine-evaluable predicate.
`gap:ambiguous-identity`	parametric	$\theta_{low} \le \sigma < \theta_{high}$ at identity bind.

gap	source class	meaning
`gap:budget-exhausted`	parametric	discharge deferred by widening.
`gap:trivial`	parametric (VoI)	neural triviality judgment — not worth discharging.
`gap:low-confidence`	parametric (VoI)	confidence below floor (`gap:confidence-ceiling` = stuck at a ceiling).

Every gap maps to v3 `gap_type` (`G-DELIVERY`, `G-VERIFY`) + `status: typed-gap`.

---

8. Promise lifecycle



A promise is sticky once grounded (§4.5). A superseded promise is an A6 diachronic change recorded as a new `Generation` — never deleted.

---

9. NN-compilable graph-export contract (**`CON-N2`**)

The graph MUST be exportable losslessly as a typed `MultiDiGraph` (Stage-1 export already lands in AMS; see `docs/graph-export.md`). Normative requirements:

- Every node kind, every relationship (with `axis` type), every Promise and Evidence sub-node, and every I/O entity MUST survive a round-trip.
- Parallel typed edges MUST be preserved as distinct edges.
- Promise provenance (owner → promise → evidence) MUST be walkable, so output provenance is itself NN-compilable supervision.
- Export MUST be deterministic (stable node/edge ordering).

This is the interface (`IF-GraphExport`, `nn-compilable-graph-export`) by which downstream consumers (`arch-agent`, the N2 compiler/trainer) read the graph.

---

10. Conformance checklist

A tool or model conforms to the openAG standard iff:

1. Every node is one of the §2 kinds; Promises, Evidence, and I/O entities are first-class nodes.
2. A1 `contains` is a strict partition; A2 `realizes` is many-to-many free.
3. Every relationship and promise carries a valid `axis`; every promise carries a `valence`.
4. Every attribute resolves to exactly one of the four end-states (§4.3); only `typed-gap` is reported as a gap.
5. Every Interface and Behavior carries a static and a dynamic promise.
6. Grounding routes structural→symbolic, semantic→neural, across the four oracles.
7. Every ungroundable demanded promise is emitted as a typed gap from §7 — never silently omitted (gap-honesty).

8. The graph round-trips losslessly through the `CON–N2` export.

Cyber-physical delta (opt-in). A graph that activates the extensions (`cyber–physical–extensions.md`) additionally conforms iff:

9. Active domain profiles are declared in `meta.domain_profiles`; every Component `subkind` is drawn from an active profile's vocabulary; profile validation rules hold.
10. Every relationship and promise carries a valid `axis` including A8; A1 (logical containment) and A8 (physical placement) are kept distinct.
11. Geometry/kinematic/live promises (§4.7) follow their typed shapes; heavy payloads are referenced by URI, never inlined; the graph still round-trips losslessly through `CON–N2`.
12. Live-stateful components expose a `live_state` dynamic promise with a binding descriptor, grounded only by the runtime oracle.

---

11. Status and non-goals

- Design-only. No code or standard artifacts exist yet; this is the contract a future implementation MUST satisfy.
- Synchronic scope. The convergent ROUND engine covers ~60% of systems engineering. Trade-space / decision-management (a diachronic meta-ROUND) and full A6 lifecycle are deferred and out of scope for this draft.
- AMS frozen until parity. No new AMS feature work proceeds until openAG reaches functional parity.
- Cyber-physical extensions are additive and design-only. Domain profiles, the A8 axis, the twin vocabulary, and the 5th oracle (`cyber–physical–extensions.md`) are opt-in hooks derived by hand-authoring the Heller Cell reference model (`docs/plans/2026–10–12–heller–cell–openAG–design.md`). No schema file is emitted yet; findings from that authoring will refine both documents.